

UNIVERSIDADE FEDERAL DE LAVRAS – UFLA

Curso: Engenharia de Controle e Automação

Disciplina: GAT108 – Automação Avançada

Turma: 22A

Docente: Arthur de Miranda Neto

Semestre: 2026/1

PLANO DE AVALIAÇÕES PRÁTICAS – PROJETO INTEGRADOR "AUTOTARGET"

1. APRESENTAÇÃO

Este documento descreve as quatro avaliações que compõem a nota final da disciplina, conforme o plano de ensino. As avaliações são baseadas em um projeto prático incremental, desenvolvido ao longo do semestre nas aulas de laboratório. O projeto consiste na implementação de um jogo para dispositivos Android chamado **AutoTarget** que servirá como plataforma para aplicação dos conceitos teóricos abordados: programação concorrente, sistemas de tempo real, reconciliação de dados, segurança cibernética, entre outros. **Deve-se instalar o Android Studio, a linguagem de programação a ser utilizada é o Java.**

Objetivo geral do jogo e descrição do cenário:

O jogo simula um sistema de defesa automatizado. Alvos (representados por círculos) surgem e se movem aleatoriamente em uma tela. O jogador pode posicionar canhões automáticos que, uma vez ativados, disparam projéteis para abater os alvos. Cada canhão opera em uma thread independente, e os alvos também são threads. O sistema deve garantir sincronização no acesso a recursos compartilhados (por exemplo, área de colisão), registrar estatísticas em banco de dados, criptografar dados sensíveis e aplicar técnicas de reconciliação de dados para corrigir medições simuladas de sensores (distância, velocidade).

Avaliações:

Este documento descreve quatro avaliações práticas (AV1, AV2, AV3 e AV4) que compõem a nota final da disciplina, conforme o plano de ensino. As avaliações são baseadas em um projeto prático incremental, desenvolvido ao longo do semestre nas aulas de laboratório. As avaliações (AV1, AV2, AV3 e AV4) seguem o cronograma da disciplina e devem ser entregues nas datas indicadas. Cada avaliação agrega novas funcionalidades ao projeto, respeitando a ordem de apresentação dos conteúdos.

2. REGRAS GERAIS PARA TODAS AS AVALIAÇÕES

- **Individualidade:** O trabalho é individual. Discussões e trocas de ideias são permitidas, mas a implementação e os relatórios devem ser próprios. Plágio resultará em nota zero.
- **Ferramentas:** Android Studio com linguagem Java. Uso obrigatório de GitHub para versionamento (incluir professor como colaborador: arthur.miranda@ufla.br).
- **Entrega:** Para cada avaliação, deverá ser enviado um e-mail para arthur.miranda@ufla.br contendo:
 - Assunto: [GAT108] Automação Avançada – AVx – Nome Completo – Turma 22A
 - Corpo: nome completo, turma, link para vídeo não listado no YouTube (máx. 15 minutos para AV1, AV2, AV3; AV4), link do repositório GitHub e relatório em PDF anexado.

- **Relatório:** Deve conter introdução, descrição das funcionalidades implementadas, explicação dos conceitos utilizados, prints das telas, trechos de código relevantes, análise de resultados e conclusão.
- **Vídeo:** Demonstração clara e prática do aplicativo em execução, explicando as principais partes do código e mostrando o cumprimento dos requisitos.
- **Confirmação:** O aluno é responsável por verificar se o professor recebeu a mensagem dentro do prazo.
- **Dúvidas:** Esclarecimentos nos horários da disciplina, fora dos horários, havendo disponibilidade do professor, mediante agendamento prévio por e-mail.
- **Use a Inteligência Artificial a vontade, mas não se esqueça que o projeto é seu e deverá demonstrar domínio completo sobre todos os seus elementos.**

3. CRONOGRAMA DE ENTREGAS E PESOS

Avaliação	Data de Entrega	Peso na Nota Final
AV1	27/04/2026	30%
AV2	25/05/2026	30%
AV3	15/06/2026	30%
AV4	29/06/2026	10%

4. INFORMAÇÕES COMPLEMENTARES

4.1. OBJETIVO PRÁTICO DAS AVALIAÇÕES

Implementar uma versão competitiva do jogo AutoTarget, onde dois sistemas independentes (esquerda e direita) disputam o controle de alvos, utilizando reconciliação de dados para posicionar canhões de forma otimizada, considerando penalidades por uso excessivo de recursos. A avaliação integra conceitos de programação concorrente, sincronização, sistemas de tempo real, reconciliação de dados e otimização.

4.2. DESCRIÇÃO DO CENÁRIO

A tela do aplicativo é dividida verticalmente em duas áreas de jogo (campo esquerdo e campo direito). Cada área possui seu próprio conjunto de canhões, controlados por threads independentes. Alvos (círculos) surgem em posições aleatórias em toda a tela, mas só podem ser abatidos pelos canhões da área em que se encontram no momento do disparo. Quando um alvo cruza a linha divisória, ele passa a pertencer à outra área.

Cada sistema (esquerda/direita) tem um orçamento de “energia” que é consumido por cada canhão ativo por segundo. O número máximo de canhões é limitado (por exemplo, 10), e canhões adicionais além de um limite base (ex.: 5) sofrem penalidade na taxa de disparo (aumento do intervalo entre disparos). O objetivo é maximizar o número de alvos abatidos ao final de um tempo pré-definido (ex.: 60 segundos). O sistema vencedor é aquele com mais abates.

A reconciliação de dados é usada para, periodicamente, corrigir as leituras dos sensores (posição e velocidade dos alvos) e, com base nessas informações, decidir se é vantajoso realocar canhões existentes ou adicionar/remover canhões, considerando a penalidade.

5. VISÃO GERAL DO PROJETO AUTOTARGET, TELAS E ARQUITETURA

O jogo simula um sistema de defesa automatizado. Alvos (círculos) surgem e se movem de forma pseudoaleatória em um canvas. O jogador pode posicionar canhões automáticos (triângulos) que, uma vez ativados, disparam projéteis para abater os alvos. Cada canhão opera em uma thread independente e os alvos também podem ser modelados como threads, exigindo sincronização no acesso a recursos compartilhados (listas e regiões críticas).

5.1. TELAS PROPOSTAS (MOCKUPS)

Os mockups abaixo são ilustrativos e funcionam como guia visual mínimo para a implementação.

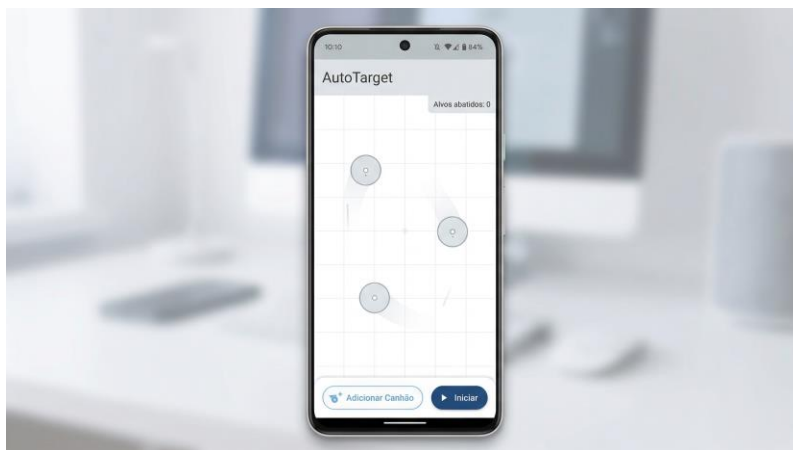


Figura – Tela principal (AV1): canvas com alvos, botões ‘Iniciar’ e ‘Adicionar Canhão’.

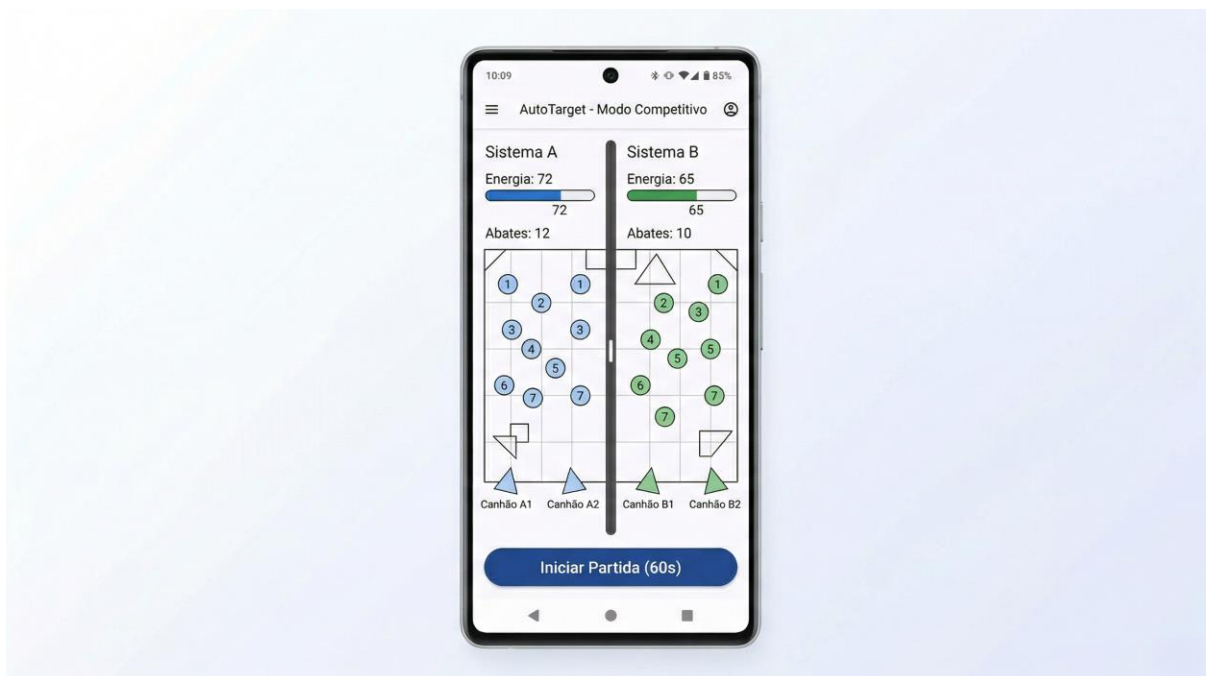


Figura – Modo competitivo (AV2): divisão por linha central, energia e abates por sistema.

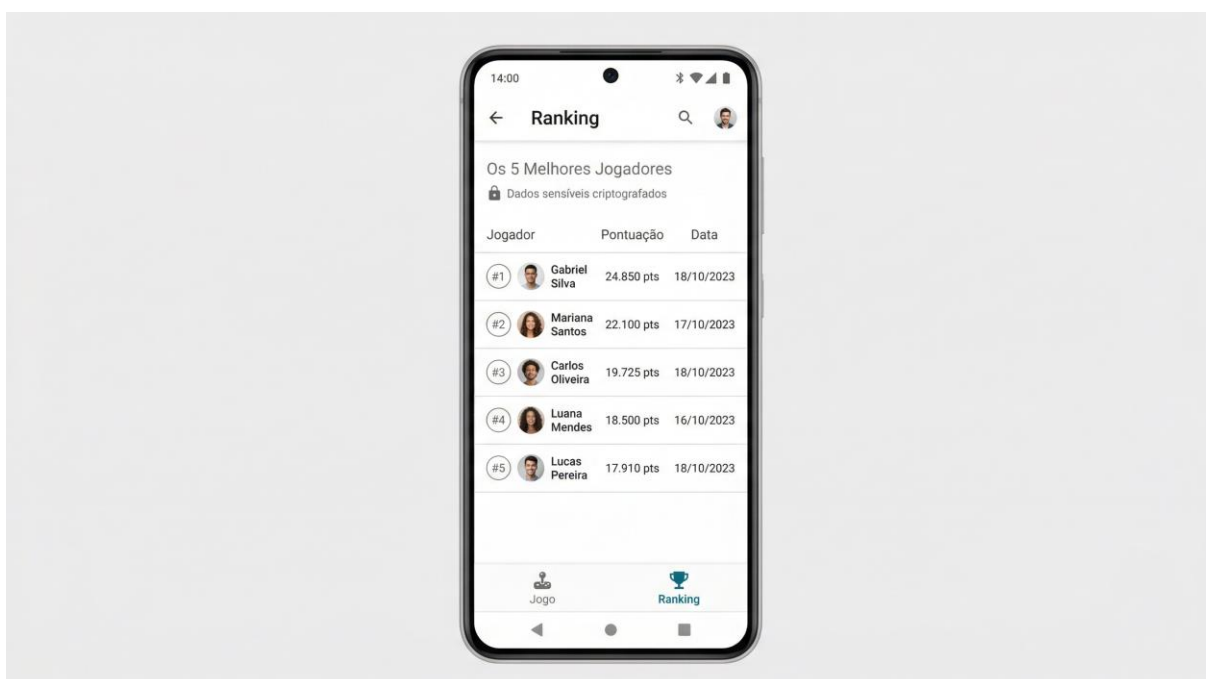


Figura – Ranking (AV3): lista de melhores pontuações e indicação de dados sensíveis criptografados.

5.2. ARQUITETURA SUGERIDA E CONCORRÊNCIA

A aplicação envolve UI (renderização) e lógica do jogo em threads, além de tarefas periódicas (coleta, reconciliação/otimização, persistência e telemetria). Os diagramas abaixo auxiliam o entendimento e a documentação no relatório.

Arquitetura e Threads do AutoTarget

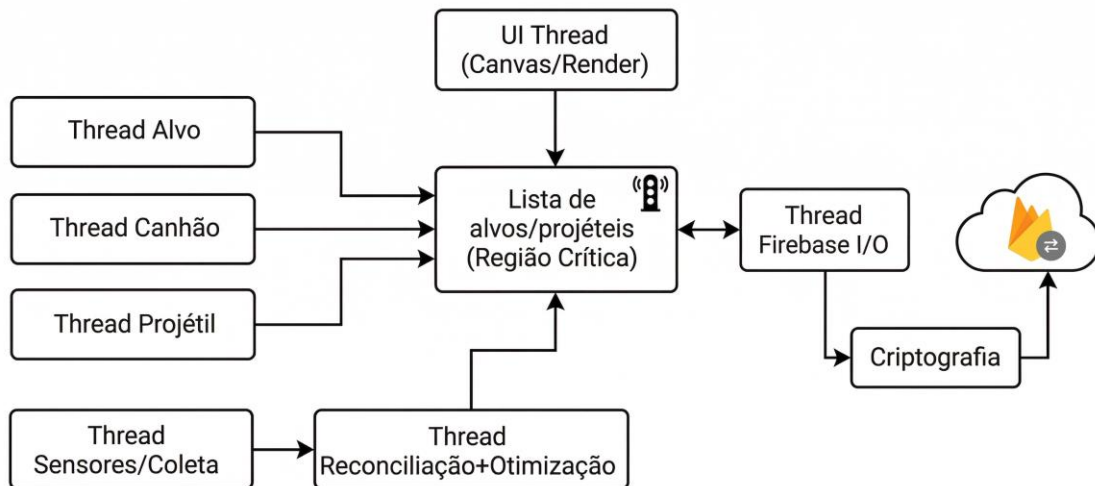


Figura – Arquitetura e threads (inclui região crítica e fluxo de dados).

Ciclo do Jogo (Loop Principal)

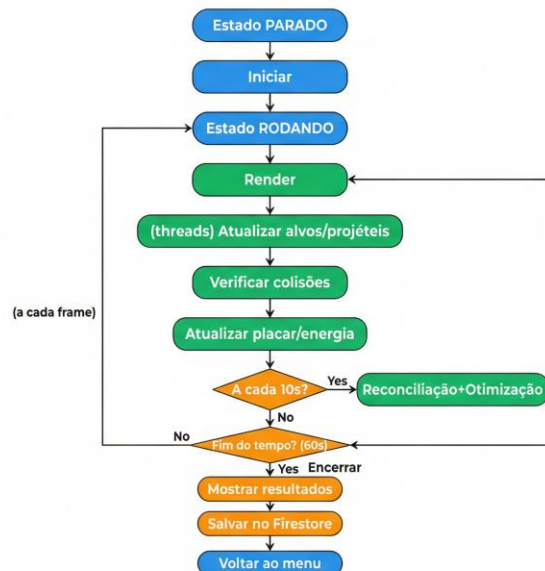


Figura – Ciclo do jogo (loop principal) e tarefas periódicas.

Classes Principais (AV1)

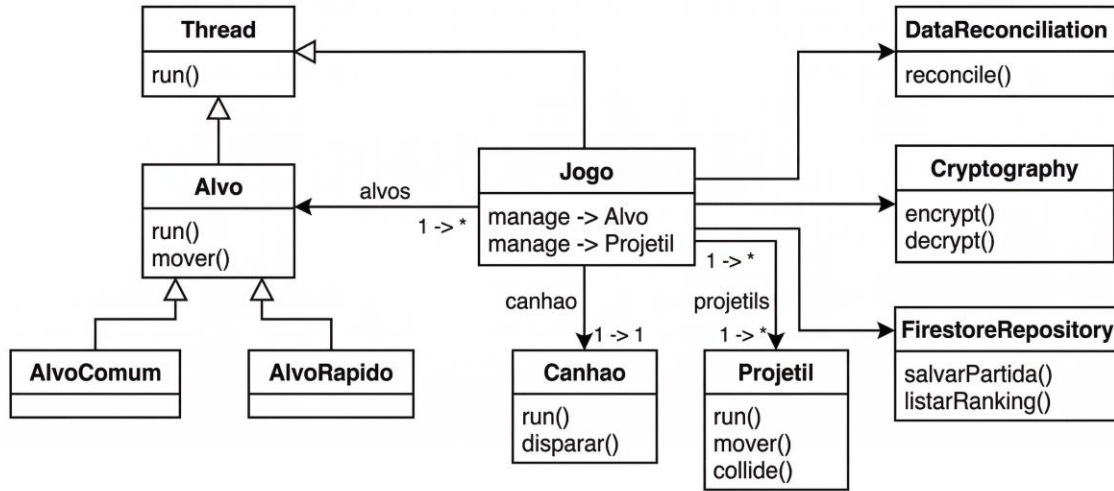


Figura – Classes principais sugeridas (UML simplificado).

6. TRANSPARÊNCIA DE CRITÉRIOS DAS AVALIAÇÕES (RUBRICAS)

As rubricas abaixo definem critérios e níveis de desempenho (Insuficiente / Adequado / Bom / Excelente) para tornar a avaliação mais transparente e favorecer feedback formativo.

6.1. AV1 – FUNDAMENTOS: THREADS, SINCRONIZAÇÃO E POO

Data de entrega: 27/04/2026

6.1.1 OBJETIVO

Implementar a base do jogo AutoTarget aplicando conceitos de programação orientada a objetos, threads, sincronização (semáforos ou synchronized), tratamento de exceções e testes unitários.

6.1.2. REQUISITOS

6.1.3. TELA PRINCIPAL (UI/INTERAÇÃO)

- Um canvas onde alvos (círculos) se movem aleatoriamente.
- Um botão "Iniciar" que dá partida no jogo.
- Um botão "Adicionar Canhão" que posiciona um canhão (representado por um triângulo) em um ponto da tela (clique do usuário ou método inteligente define a posição).

6.1.4. CLASSES E THREADS

- Classe Alvo (Thread): atributos: posição (x, y), raio, velocidade, ativo. Move-se (ponto a ponto na tela). Método run() atualiza posição e verifica colisões com projéteis.
- Classe Canhão (Thread): atributos: posição, ângulo, lista de projéteis. Método run() cria e dispara projéteis periodicamente. Cada canhão, quando ativado, dispara automaticamente projéteis em direção aos alvos mais próximos (thread separada).
- Classe Projétil (Thread): atributos: posição, direção, velocidade. Move-se (ponto a ponto na tela) em linha reta e verifica colisão com alvos.
- Classe Jogo (gerencia as listas de alvos, canhões e projéteis, controla o loop principal da UI).

6.1.5. SINCRONIZAÇÃO

- Usar semáforos (ou synchronized) para controlar o acesso à lista de alvos e projéteis, garantindo que apenas uma thread modifique a lista por vez.
- Implementar uma região crítica onde apenas um projétil pode verificar colisão com um alvo por vez, evitando condições de corrida.

6.1.6. TRATAMENTO DE EXCEÇÕES

- Incluir blocos try-catch para tratar possíveis exceções (por exemplo, divisão por zero ao calcular ângulo, índices fora da lista).
- Criar uma exceção personalizada JogoException e lançá-la em situações de erro (ex.: tentar adicionar canhão fora dos limites da tela).

6.1.7. TESTES UNITÁRIOS

- Criar pelo menos três testes JUnit para métodos críticos (ex.: cálculo de distância entre pontos, verificação de colisão, validação de posição).

6.1.8. POLIMORFISMO E HERANÇA

- Criar hierarquia de alvos: AlvoComum e AlvoRapido (subclasses) sobrescrevendo movimento.
- Demonstrar polimorfismo ao percorrer lista de alvos e chamar o método de movimento adequado.

6.1.9. RUBRICA DE AVALIAÇÃO (AV1)

Critério	Peso	Insuficiente	Adequado	Bom	Excelente
Tela principal e interação	20%	UI incompleta ou não funcional.	UI funcional com falhas menores.	UI completa e estável.	UI excelente + feedback visual (estado/placar/logs) e organização.
Classes/threads e funcionamento	25%	Modelagem inconsistente; threads não	Funciona parcialmente.	Funciona corretamente e de forma	Código limpo, extensível e bem modular.

		funcionam.		consistente.	
Sincronização e região crítica	20%	Condições de corrida frequentes; travamentos.	Sincronização mínima; inconsistências ocasionais.	Sincronização correta e estável.	Além do correto: evidências (logs/testes) e justificativas.
Tratamento de exceções (inclui JogoException)	10%	Sem tratamento; crashes.	Tratamento superficial.	Tratamento adequado.	Tratamento completo + mensagens úteis + validações preventivas.
Testes unitários (JUnit)	10%	Sem testes.	Testes insuficientes.	≥3 testes cobrindo pontos críticos.	≥3 testes + bons casos de borda e organização.
Herança e polimorfismo	15%	Não implementa.	Implementa mas não demonstra bem.	Implementa e demonstra no fluxo.	Design bem justificado + extensibilidade evidente.

6.1.10. ENTREGÁVEIS (AV1)

- Repositório GitHub com código e histórico de commits.
- Relatório PDF (recomendado até 15 páginas) com prints e trechos de código.
- Vídeo (máx. 15 min) demonstrando execução e explicando o código.

6.2 AV2 – COMPETIÇÃO, RECONCILIAÇÃO DE DADOS E OTIMIZAÇÃO

Data de entrega: 25/05/2026

6.2.1. OBJETIVO

Implementar uma versão competitiva do jogo AutoTarget, onde dois sistemas independentes (esquerda e direita) disputam o controle de alvos, utilizando reconciliação de dados para posicionar canhões de forma otimizada, considerando penalidades por uso excessivo de recursos. A avaliação integra conceitos de programação concorrente, sincronização, sistemas de tempo real, reconciliação de dados e otimização.

6.2.2 REQUISITOS E TÉCNICOS

a) Divisão de Tela e Controle de Alvos:

- A tela deve ser dividida por uma linha vertical no centro.
- Cada lado possui uma lista de canhões (threads) que só podem atirar em alvos cujo centro esteja na sua metade.

- Implementar um mecanismo para detectar quando um alvo cruza a linha: nesse momento, ele é transferido da lista do lado antigo para o novo. A transferência deve ser atômica (usar synchronized ou semáforo).
- Ao final do tempo, exibir o placar (número de abates de cada lado) e declarar o vencedor.

b) Modelo de Recursos e Penalidades:

- Definir um orçamento inicial de energia para cada lado (ex.: 100 unidades).
- Cada canhão ativo consome 1 unidade de energia por segundo (ou por ciclo de disparo). A energia é decrementada em tempo real.
- Se a energia chegar a zero, os canhões desse lado param de disparar.
- Penalidade por excesso de canhões: se o número de canhões em um lado for maior que um limite L (ex.: 5), o intervalo de disparo de cada canhão nesse lado é multiplicado por um fator $(1 + (n - L) * 0.2)$, ou seja, aumenta 20% por canhão extra.
- O aluno pode escolher outros modelos de penalidade, desde que justificados no relatório.

c) Sensores Simulados e Coleta de Dados:

- Cada alvo possui um sensor virtual que fornece leituras ruidosas de sua posição (x, y) e velocidade (vx, vy). O ruído é gaussiano com média zero e desvio padrão proporcional ao valor real (ex.: 5%).
- A cada segundo, cada lado coleta as leituras dos alvos que estão em seu território e armazena em um buffer (pelo menos 10 leituras por alvo para análise).

d) Reconciliação de Dados para Otimização de Posicionamento:

- Implementar a classe DataReconciliation com o método reconcile(double[] y, double[][] V, double[][] A) que retorna o vetor reconciliado $\hat{y}$, conforme:

$$\hat{y} = y - VA^T(AVA^T)^{-1}Ay$$

- Modelar o problema de posicionamento como um sistema de fluxos: cada alvo i tem uma “demanda” (probabilidade de ser abatido) que depende da distância aos canhões. Os canhões j têm uma “capacidade” (taxa de disparo). A reconciliação é usada para estimar a matriz de incidência A que relaciona alvos e canhões, a partir das leituras ruidosas de distância.
- A cada 10 segundos, cada lado deve:
 - Reunir todas as leituras dos últimos 10 segundos para cada alvo.

- Calcular a média e a variância das distâncias entre cada canhão e cada alvo (construindo a matriz de covariância V).
- Definir a matriz de incidência A baseada na geometria (ex.: se a distância média for menor que um limiar, há conexão).
- Aplicar a reconciliação para obter distâncias corrigidas.
- Com base nas distâncias reconciliadas, calcular uma posição “ótima” para cada canhão (por exemplo, o centroide dos alvos que estão mais próximos). Se a nova posição for significativamente diferente da atual, o canhão deve se mover gradualmente (simulando realocação) – isso pode ser feito atualizando sua posição na tela.

e) Decisão de Adicionar/Remover Canhões com Base em Custo-Benefício:

- A cada ciclo de reconciliação, cada lado avalia se deve aumentar ou diminuir o número de canhões.
- Para isso, deve-se estimar o ganho esperado em abates com a adição de um novo canhão (baseado nas distâncias reconciliadas e na taxa de disparo) e comparar com o custo adicional de energia e a penalidade por excesso.
- Implementar uma função de utilidade que retorne o número esperado de abates por segundo dada uma configuração de canhões. A decisão pode ser tomada por um algoritmo guloso ou por otimização simples.
- A adição/remoção de canhões deve ser feita criando ou interrompendo threads, com cuidado para evitar condições de corrida.

f) Escalonamento de Tempo Real e Análise de Escalonabilidade:

- Identificar as tarefas (threads) do sistema competitivo, incluindo:
 - T1: Movimentação dos alvos (várias threads, uma por alvo)
 - T2: Disparo dos canhões de cada lado (várias threads)
 - T3: Verificação de colisões
 - T4: Atualização da UI
 - T5: Coleta de dados dos sensores
 - T6: Execução da reconciliação e otimização (periódica)
 - T7: Gerenciamento de energia e penalidades
 - (Pelo menos 8 tarefas no total)
- Definir para cada tarefa: prioridade, período (P_i), tempo de execução (C_i), deadline (D_i) e jitter (J_i) – valores coerentes com a simulação.

- Construir o grafo de dependências (se houver) e preencher a tabela de tarefas.
- Calcular o tempo de resposta máximo R_i para cada tarefa usando a equação de resposta com prioridades fixas (Rate Monotonic) e verificar se o conjunto é escalonável em um único processador.
- **Variação de processadores:** Usar `Process.setThreadAffinityMask()` para executar as threads em 1, 2 e todos os núcleos disponíveis. Medir os tempos de resposta e verificar em quais configurações ocorre perda de escalonabilidade.
- Apresentar gráficos comparativos e conclusões.

6.2.3. RUBRICA DE AVALIAÇÃO (AV2)

Critério	Peso	Insuficiente	Adequado	Bom	Excelente
Divisão de tela, pertencimento e placar	15%	Regras incorretas; placar instável.	Funciona com falhas em casos limite.	Correto e estável.	Correto + excelente apresentação e testes de borda.
Modelo de energia e penalidade	15%	Ausente/incoerente.	Implementa parcialmente.	Implementa corretamente e justifica.	Além do básico: logs/gráficos mostrando impacto.
Sensores ruidosos + buffers	10%	Sem sensores/buffer.	Sensores ou buffer incompletos.	Implementação correta.	Inclui análise estatística (média/variação) bem apresentada.
Reconciliação de dados (implementação + evidência)	25%	Não aplica ou não evidencia.	Aplica sem contrato claro/sem comparação.	Aplica com comparação antes/depois.	Validação quantitativa (erro/variação) + explicação didática.
Otimização (mover/adicionar/remover)	15%	Sem estratégia coerente.	Estratégia simples e instável.	Estratégia coerente com melhoria observável.	Função utilidade clara + comparação de desempenho e justificativa.
Tempo real: tarefas e análise	20%	Não identifica tarefas/tempos.	Tabela incompleta.	Tabela coerente + análise.	Medições reais + gráficos + discussão de gargalos.

6.2.4. ENTREGÁVEIS (AV2)

- Código completo no GitHub.
- Relatório PDF (máx. 15 páginas) com: reconciliação, custo-benefício, tabela de tarefas e resultados.
- Vídeo (máx. 15 min) demonstrando competição, valores antes/depois e decisões do sistema.

6.3. AV3 – Banco de dados, criptografia e sistemas ciberfísicos

Data de entrega: 15/06/2026

6.3.1. OBJETIVO

Persistir dados do jogo no Firebase (Firestore), aplicar criptografia em dados sensíveis, e explorar um sistema ciberfísico por meio de telemetria e controle por feedback.

6.3.2. REQUISITOS FUNCIONAIS

a) Integração com Firebase Firestore:

- Criar uma coleção partidas para armazenar o histórico de jogos: timestamp, pontuação final, número de alvos abatidos, configurações dos canhões.
- Ao final de cada partida, salvar esses dados no Firestore.
- Implementar uma tela de ranking que exibe as melhores pontuações (ordenadas) lidas do banco.

b) Criptografia:

- Criar uma classe Cryptography com métodos encrypt e decrypt usando AES (chave fixa ou derivada de senha).
- Antes de enviar os dados para o Firebase, criptografar campos sensíveis (ex.: nome do jogador, pontuação) em formato JSON.
- Ao ler, descriptografar e exibir.
- Armazenar no banco apenas os dados criptografados.

c) Threads para operações de banco de dados:

- Todas as operações de leitura/escrita no Firebase devem ser feitas em threads separadas (não na UI thread). Usar AsyncTask ou ExecutorService.
- Garantir sincronização para evitar conflitos de leitura/escrita simultânea.

d) Sistemas ciberfísicos:

- Simular um sensor de temperatura do dispositivo (usar SensorManager para obter temperatura da bateria ou criar um valor aleatório).

- A cada 10 segundos, registrar a temperatura em uma coleção telemetria no Firebase, com timestamp.
- Se a temperatura ultrapassar um limiar (ex.: 40°C), o sistema deve reduzir a taxa de disparo dos canhões (controle feedback) para evitar superaquecimento – caracterizando um sistema ciberfísico.

e) Testes de segurança:

- Implementar autenticação simples (login/senha) usando Firebase Authentication.
- Cada usuário tem seu próprio conjunto de partidas.
- Garantir que um usuário não possa acessar dados de outro (regras de segurança no Firestore).

6.3.3. CRITÉRIOS ADICIONAIS

- O código deve demonstrar o uso correto de promessas/callbacks do Firebase.
- Incluir no relatório a explicação do algoritmo de criptografia e as políticas de segurança.

6.3.4. RUBRICA DE AVALIAÇÃO (AV3)

Critério	Peso	Insuficiente	Adequado	Bom	Excelente
Firestore: persistência + ranking	20%	Não salva/lê corretamente.	Salva ou lê parcialmente.	Salva e lista ranking corretamente.	Estrutura de dados limpa + ordenação/paginação consistente.
Criptografia (AES) em dados sensíveis	25%	Sem criptografia.	Criptografa com falhas.	Criptografa/descriptografa corretamente.	Implementação clara + tratamento de erros + explicação correta.
Threads para operações de banco	15%	Roda em UI thread.	Parte fora da UI thread.	Tudo fora da UI thread; sem travamentos.	Arquitetura robusta (repo/service) + logs e sincronização adequada.
Sistema ciberfísico (telemetria + feedback)	20%	Não implementa.	Implementa apenas telemetria ou apenas feedback.	Implementa ambos e demonstra efeito.	Inclui análise (gráfico) e discussão de estabilidade/limiar.
Segurança: auth + isolamento por usuário	20%	Sem auth/regras.	Auth básico sem regras claras.	Auth + regras corretas e evidência do isolamento.	Documentação das regras + validações e tratamento de sessão.

6.3.5. ENTREGÁVEIS (AV3)

- Código no GitHub com integração Firebase.
- Relatório PDF (máx. 15 páginas) com: modelo de dados, criptografia e regras de segurança.
- Vídeo (máx. 15 min) mostrando ranking, dados criptografados e telemetria/feedback.

6.4. AV4 – TRL E ANÁLISE DE DESEMPENHO (LEI DE AMDAHL)

Data de entrega: 29/06/2026

6.4.1. OBJETIVO

Avaliar o projeto sob a ótica de níveis de maturidade tecnológica (TRL) e realizar uma análise de desempenho e escalabilidade (Lei de Amdahl), propondo e implementando ao menos uma melhoria.

6.4.2. REQUISITOS

a) **Classificação TRL:**

- Elaborar um documento (parte do relatório) classificando o projeto AutoTarget nos níveis TRL de 1 a 9, justificando cada nível com base nas funcionalidades implementadas (ex.: TRL 4 – validação em ambiente laboratorial, TRL 6 – demonstração em ambiente relevante, etc.).

b) **Análise de desempenho com Lei de Amdahl (30%):**

- Executar o jogo variando o número de threads de alvos (10, 20, 50, 100) e o número de núcleos disponíveis (via afinidade).
- Medir o tempo total de processamento de um ciclo de jogo (ex.: 30 segundos) e calcular o speedup em relação à execução com uma thread.
- Aplicar a Lei de Amdahl:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

onde P é a fração paralelizável.

- Estimar P a partir dos dados experimentais e discutir as discrepâncias.

c) **Otimização e escalonabilidade (25%):**

- Com base nos resultados, propor melhorias no escalonamento das threads (ex.: ajuste de prioridades, uso de pool de threads) para aumentar a escalonabilidade.

- Implementar pelo menos uma melhoria e comparar os resultados antes/depois.

d) Relatório final e vídeo de apresentação:

- Consolidar todas as análises das quatro avaliações em um relatório final, incluindo gráficos, tabelas e conclusões.
- Gravar um vídeo de até 10 minutos apresentando o projeto completo, destacando as principais funcionalidades, os desafios enfrentados e os resultados das análises.

6.4.3. RUBRICA DE AVALIAÇÃO (AV3)

Critério	Peso	Insuficiente	Adequado	Bom	Excelente
Classificação TRL e justificativas	20%	Incoerente ou sem justificativas.	Justificativas superficiais.	Justificativas coerentes.	Justificativas bem fundamentadas + reflexão crítica de limitações.
Experimentos + Amdahl (métricas e análise)	30%	Sem medições.	Medições incompletas.	Medições coerentes + speedup + estimativa de P.	Gráficos claros + discussão de gargalos e discrepâncias.
Otimização e escalabilidade (proposta)	25%	Sem proposta.	Proposta genérica.	Proposta específica e alinhada aos dados.	Proposta com impacto esperado, riscos e plano de validação.
Implementação da melhoria (antes/depois)	10%	Não implementa.	Implementa sem evidências.	Implementa com comparação.	Comparação rigorosa (mesma config, repetição, variância).
Relatório final e vídeo	15%	Incompleto.	Completo, porém desorganizado.	Completo e claro.	Narrativa excelente + figuras/tabelas e conclusões sólidas.

6.4.4. ENTREGÁVEIS (AV4)

- O relatório final deve ser enviado juntamente com o link do GitHub e do vídeo, conforme regras gerais.

- A nota da AV4 será composta pela qualidade do relatório (15%) e pela implementação das melhorias propostas (10%).

Exemplo do uso da Inteligência Artificial (IA) em parte do problema, a você de averiguar e se certificar. Estamos na Automação Avançada, seria um contrassenso não utilizar de recursos tão poderosos. Você precisa estar treinado para usar ferramentas poderosas a seu favor, contudo, a decisão final e a responsabilidade pelo resultado e de como foi obtido será sempre seu. Atenção, eu fiz apenas um copy/paste e solicitei para a IA propor uma solução.

Solução Completa para Reconciliação de Dados no Projeto AutoTarget (AV2)

1. Objetivo da Reconciliação de Dados

A reconciliação de dados é utilizada para corrigir medições ruidosas de distância entre alvos e canhões, impondo restrições lineares que derivam da geometria do sistema. Após a reconciliação, obtemos distâncias consistentes que permitem estimar com maior precisão a posição dos alvos e, conseqüentemente, otimizar o posicionamento dos canhões, bem como decidir sobre a adição ou remoção de canhões com base em custo-benefício.

2. Definição do Problema

Considere um dos lados da tela (esquerdo ou direito) com:

- **N** canhões em posições conhecidas $(x_j, y_j), j = 1, \dots, N$.
- **M** alvos ativos, cada um com posição desconhecida $(X_i, Y_i), i = 1, \dots, M$.

A cada ciclo de 10 segundos, para cada par (alvo i , canhão j) dispomos de um conjunto de medições de distância obtidas ao longo do intervalo. A partir dessas medições, calculamos:

- A **média amostral** $\bar{d}_{ij}$ das distâncias.
- A **variância amostral** s_{ij}^2 das distâncias.

Desejamos obter estimativas reconciliadas $\hat{d}_{ij}$ que respeitem as relações geométricas impostas pelas posições fixas dos canhões.

3. Construção do Vetor de Medidas e Matriz de Covariância

Para linearizar o problema, trabalhamos com os **quadrados das distâncias ajustados**:

$$y_{ij} = \bar{d}_{ij}^2 - (x_j^2 + y_j^2).$$

O termo $(x_j^2 + y_j^2)$ é conhecido. O vetor global $\mathbf{y}$ tem dimensão $M \times N$, organizado como:

$$\mathbf{y} = [y_{11}, y_{12}, \dots, y_{1N}, y_{21}, \dots, y_{MN}]^T.$$

A matriz de covariância $\mathbf{V}$ é diagonal, com entradas:

$$\mathbf{V}_{(i,j),(i,j)} = \text{Var}(y_{ij}) \approx (2\bar{d}_{ij})^2 s_{ij}^2,$$

onde usamos a aproximação de delta method para a variância de $\bar{d}_{ij}^2$. Caso as medições sejam independentes entre si, essa diagonal é adequada.

4. Restrições Lineares Derivadas da Geometria dos Canhões

Para um alvo fixo (X_i, Y_i) , a distância verdadeira ao canhão j satisfaz:

$$d_{ij}^2 = (X_i - x_j)^2 + (Y_i - y_j)^2 = (X_i^2 + Y_i^2) - 2X_i x_j - 2Y_i y_j + (x_j^2 + y_j^2).$$

Subtraindo $(x_j^2 + y_j^2)$ de ambos os lados, obtemos:

$$y_{ij}^{\text{true}} = d_{ij}^2 - (x_j^2 + y_j^2) = K_i - 2X_i x_j - 2Y_i y_j,$$

onde $K_i = X_i^2 + Y_i^2$. Assim, para um dado alvo i , o vetor $\mathbf{y}_i^{\text{true}} = [y_{i1}^{\text{true}}, \dots, y_{iN}^{\text{true}}]^T$ pertence ao espaço coluna da matriz $\mathbf{M}$ (de dimensão $N \times 3$):

$$\mathbf{M} = \begin{bmatrix} 1 & -2x_1 & -2y_1 \\ 1 & -2x_2 & -2y_2 \\ \vdots & \vdots & \vdots \\ 1 & -2x_N & -2y_N \end{bmatrix}.$$

Ou seja, existe um vetor $\boldsymbol{\theta}_i = [K_i, X_i, Y_i]^T$ tal que $\mathbf{y}_i^{\text{true}} = \mathbf{M}\boldsymbol{\theta}_i$.

Consequentemente, $\mathbf{y}_i^{\text{true}}$ deve ser ortogonal a qualquer vetor no espaço nulo esquerdo de $\mathbf{M}$. Seja $\mathbf{C}$ uma matriz cujas linhas formam uma base do espaço nulo esquerdo de $\mathbf{M}$ (dimensão $N - 3$, assumindo canhões não colineares). Então:

$$\mathbf{C} \mathbf{y}_i^{\text{true}} = \mathbf{0}.$$

Para todos os alvos simultaneamente, definimos a matriz de restrições global $\mathbf{A}$ (bloco diagonal):

$$\mathbf{A} = \begin{bmatrix} \mathbf{C} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{C} & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{C} \end{bmatrix},$$

com dimensão $(M \cdot (N - 3)) \times (M \cdot N)$. A condição de reconciliação é $\mathbf{A}\mathbf{y}^{\text{true}} = \mathbf{0}$.

Exemplo com $N = 3$ canhões

Se $N = 3$ e os canhões não são colineares, $\mathbf{M}$ é 3×3 e tem posto 3, logo não há espaço nulo esquerdo ($N - 3 = 0$). Nesse caso, não há restrição linear homogênea – as três distâncias determinam unicamente a posição do alvo. **A reconciliação não seria necessária para consistência.** Na prática, com $N = 3$ podemos estimar a posição diretamente. **Porém, o problema exige o uso da reconciliação, então consideraremos $N \geq 4$ para que existam restrições. O texto menciona "número máximo de canhões é limitado (por exemplo, 10)", então podemos ter $N \geq 4$.**

Exemplo com $N = 4$ canhões

Suponha quatro canhões em posições $(x_1, y_1), \dots, (x_4, y_4)$. Calculamos $\mathbf{M}$ (4×3). Seu espaço nulo esquerdo tem dimensão 1. Encontramos um vetor $\mathbf{c} = [c_1, c_2, c_3, c_4]^T$ tal que $\mathbf{c}^T \mathbf{M} = \mathbf{0}$. Isso pode ser obtido, por exemplo, calculando o determinante ou usando decomposição QR. Para cada alvo i , a restrição é $\mathbf{c}^T \mathbf{y}_i = 0$. Assim, $\mathbf{C}$ é uma matriz 1×4 . A matriz $\mathbf{A}$ global terá M linhas (uma por alvo) e $4M$ colunas, com cada linha contendo $\mathbf{c}^T$ na posição correspondente ao alvo.

5. Aplicação da Fórmula de Reconciliação

A fórmula padrão para reconciliação de dados com restrições lineares $\mathbf{A}\mathbf{y} = \mathbf{0}$ e matriz de covariância $\mathbf{V}$ é:

$$\hat{\mathbf{y}} = \mathbf{y} - \mathbf{V}\mathbf{A}^T(\mathbf{A}\mathbf{V}\mathbf{A}^T)^{-1}\mathbf{A}\mathbf{y}.$$

Devido à estrutura bloco-diagonal de $\mathbf{A}$ e $\mathbf{V}$, podemos processar cada alvo independentemente, reduzindo o custo computacional. Para um alvo i , seja $\mathbf{y}_i$ o vetor de suas N medidas ajustadas e $\mathbf{V}_i$ a submatriz diagonal correspondente. A matriz de restrições para esse alvo é $\mathbf{C}$. Então:

$$\hat{\mathbf{y}}_i = \mathbf{y}_i - \mathbf{V}_i \mathbf{C}^T (\mathbf{C} \mathbf{V}_i \mathbf{C}^T)^{-1} \mathbf{C} \mathbf{y}_i.$$

Note que $\mathbf{CV}_i\mathbf{C}^T$ é uma matriz quadrada de tamanho $(N - 3) \times (N - 3)$, invertível se $\mathbf{C}$ tiver posto linha completo e $\mathbf{V}_i$ for positiva definida.

6. Obtenção das Distâncias Reconciliadas e Posições dos Alvos

Após calcular $\hat{\mathbf{y}}_i$, obtemos as distâncias reconciliadas:

$$\hat{d}_{ij} = \sqrt{\hat{y}_{ij} + (x_j^2 + y_j^2)},$$

garantindo que o radicando seja positivo (caso contrário, pode-se usar o valor absoluto ou truncar).

Agora, para cada alvo i , podemos estimar sua posição $(\hat{X}_i, \hat{Y}_i)$ resolvendo o sistema linear proveniente da definição de $\hat{y}_{ij}$:

$$\hat{y}_{ij} = \hat{K}_i - 2\hat{X}_i x_j - 2\hat{Y}_i y_j, j = 1, \dots, N,$$

onde $\hat{K}_i = \hat{X}_i^2 + \hat{Y}_i^2$. Esse é um sistema linear nas incógnitas $\hat{K}_i, \hat{X}_i, \hat{Y}_i$. Escrevendo na forma matricial:

$$\mathbf{M} \begin{bmatrix} \hat{K}_i \\ \hat{X}_i \\ \hat{Y}_i \end{bmatrix} = \hat{\mathbf{y}}_i.$$

Como $\mathbf{M}$ é $N \times 3$, a solução de mínimos quadrados é:

$$\begin{bmatrix} \hat{K}_i \\ \hat{X}_i \\ \hat{Y}_i \end{bmatrix} = (\mathbf{M}^T \mathbf{M})^{-1} \mathbf{M}^T \hat{\mathbf{y}}_i.$$

Observe que $\mathbf{M}$ é constante (depende apenas das posições dos canhões) e pode ser pré-calculada. As coordenadas do alvo são $\hat{X}_i, \hat{Y}_i$ (a componente $\hat{K}_i$ é redundante, mas pode ser usada para verificar consistência).

7. Otimização do Posicionamento dos Canhões

Com as posições estimadas dos alvos, podemos realocar os canhões para maximizar a eficiência de abate. Uma estratégia simples é mover cada canhão para o centroide dos alvos que estão mais próximos dele, ponderado pela “demanda” (probabilidade de abate). Defina para cada canhão j um conjunto de alvos cuja distância reconciliada $\hat{d}_{ij}$ seja menor que um limiar (por exemplo, o raio de alcance).

A nova posição do canhão j pode ser:

$$\mathbf{p}_j^{\text{novo}} = \frac{\sum_{i \in \mathcal{N}_j} w_{ij} (\hat{X}_i, \hat{Y}_i)}{\sum_{i \in \mathcal{N}_j} w_{ij}},$$

onde w_{ij} é um peso inversamente proporcional à distância (ex.: $1/\hat{d}_{ij}$) ou baseado na probabilidade de acerto. Se a nova posição diferir significativamente da atual, o canhão deve mover-se gradualmente (atualizando sua posição na tela a cada frame).

8. Decisão de Adicionar/Remover Canhões (Custo-Benefício)

A cada ciclo de reconciliação, avaliamos se o número atual de canhões N é ótimo. Para isso, definimos uma **função de utilidade** $U(N)$ que estima o número esperado de abates por segundo dada uma configuração com N canhões, considerando:

- A energia disponível e o consumo por canhão.
- A penalidade por excesso de canhões (aumento do intervalo de disparo).
- A cobertura dos alvos baseada nas distâncias reconciliadas.

Modelo de Utilidade

Seja E a energia restante do lado. Cada canhão ativo consome e unidades por segundo. A taxa de disparo efetiva de um canhão é $r(N) = r_0 / (1 + \alpha \max(0, N - L))$, onde L é o limite sem penalidade e α o fator de penalidade (ex.: 0.2). O número esperado de abates por segundo de um canhão j é proporcional à soma sobre os alvos da probabilidade de acerto, que pode ser modelada como $p_{ij} = \exp(-\beta \hat{d}_{ij})$ (quanto menor a distância, maior a chance). Assim, a taxa de abates total é:

$$U(N) = \sum_{j=1}^N r(N) \sum_{i=1}^M p_{ij}.$$

No entanto, essa expressão não considera a competição entre canhões pelo mesmo alvo (um alvo só pode ser abatido uma vez). Uma aproximação mais realista é usar um modelo de alocação onde cada alvo é atribuído ao canhão mais próximo, e a taxa de abate é limitada pela capacidade do canhão. Podemos simplificar usando a soma das probabilidades, mas é aceitável para decisão relativa.

Algoritmo de Decisão

Para decidir se adicionamos um canhão, calculamos $U(N + 1)$ considerando a melhor posição para o novo canhão (por exemplo, no centroide dos alvos menos cobertos). Comparamos com $U(N)$ e também com o custo de energia adicional. Se $U(N + 1) - U(N) > \text{limiar}$, adicionamos. Analogamente para remoção: se remover um canhão (o menos útil) resulta em $U(N - 1)$ não muito menor que $U(N)$, e a economia de energia é significativa, removemos.

A implementação deve criar ou interromper threads de canhões com cuidado, usando mecanismos de sincronização (synchronized) para evitar condições de corrida ao modificar a lista de canhões.

9. Exemplo Numérico (Ilustrativo)

Suponha dois alvos ($M = 2$) e quatro canhões ($N = 4$) com posições:

- Canhão 1: (0,0)
- Canhão 2: (10,0)
- Canhão 3: (0,10)
- Canhão 4: (10,10)

Medições simuladas (após cálculo de médias e variâncias) fornecem as distâncias médias:

Alvo	Canhão	d_{ij}	s_{ij}^2
1	1	5.1	0.1
1	2	7.2	0.2
1	3	8.3	0.15
1	4	10.4	0.25
2	1	6.2	0.12
2	2	4.3	0.18

Alvo	Canhão	d_{ij}	s_{ij}^2
2	3	9.1	0.22
2	4	5.8	0.14

Passo 1: Calcular $y_{ij} = d_{ij}^2 - (x_j^2 + y_j^2)$. Por exemplo, para canhão 1 (0,0): $x_1^2 + y_1^2 = 0$, então $y_{11} = 5.1^2 = 26.01$. Para canhão 2 (10,0): $10^2 + 0^2 = 100$, $y_{12} = 7.2^2 - 100 = 51.84 - 100 = -48.16$, etc.

Passo 2: Matriz de covariância V_i para cada alvo: diagonal com $(2d_{ij})^2 s_{ij}^2$. Exemplo para alvo 1, canhão 1: $(2 * 5.1)^2 * 0.1 = (10.2)^2 * 0.1 = 104.04 * 0.1 = 10.404$.

Passo 3: Determinar C para os quatro canhões. Construímos M :

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 1 & -20 & 0 \\ 1 & 0 & -20 \\ 1 & -20 & -20 \end{bmatrix}.$$

O espaço nulo esquerdo tem dimensão 1. Resolvendo $c^T M = 0$, encontramos $c = [1, -1, -1, 1]^T$ (verifique: $1*1 + (-1)*1 + (-1)*1 + 1*1 = 0$; para a segunda coluna: $1*0 + (-1)*(-20) + (-1)*0 + 1*(-20) = 20 - 20 = 0$; terceira: $1*0 + (-1)*0 + (-1)*(-20) + 1*(-20) = 20 - 20 = 0$). Portanto, a restrição para cada alvo é: $y_{i1} - y_{i2} - y_{i3} + y_{i4} = 0$.

Passo 4: Aplicar reconciliação para cada alvo. Para alvo 1, temos $y_1 = [26.01, -48.16, -31.11, 8.16]^T$ (valores aproximados). V_1 diagonal com entradas calculadas. Calculamos $Cy_1 = 26.01 - (-48.16) - (-31.11) + 8.16 = 26.01 + 48.16 + 31.11 + 8.16 = 113.44$. O termo de correção $V_1 C^T (C V_1 C^T)^{-1} C y_1$ resulta em um vetor que subtraímos de y_1 para obter $\hat{y}_1$. (O cálculo numérico completo é extenso, mas ilustra o processo.)

Passo 5: Após obter $\hat{y}_1$, calculamos as distâncias reconciliadas e, em seguida, a posição do alvo via mínimos quadrados.

10. Considerações de Implementação

- A matriz C pode ser pré-calculada sempre que os canhões se movem (o que ocorre a cada ciclo de otimização). Como o movimento é gradual, pode-se recalculá-la a cada ciclo.
- Para evitar inversão de matrizes grandes, processe cada alvo separadamente.

- A função de utilidade deve ser implementada com base nas distâncias reconciliadas e nas posições estimadas dos alvos.
- A adição/remoção de canhões deve ser feita em uma região crítica (synchronized) para evitar que threads de disparo acessem listas inconsistentes.

11. Conclusão

Este modelo fornece uma base matemática para a reconciliação de dados no contexto do jogo AutoTarget. As distâncias corrigidas permitem estimar posições dos alvos com maior precisão, possibilitando decisões de posicionamento e dimensionamento da frota de canhões baseadas em critérios objetivos de custo-benefício. A implementação cuidadosa das threads e da sincronização garantirá a robustez do sistema competitivo.